COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/DATA STRUCTURE AND ALGORITHMS	**SECTION:**	CPE21S2
DATE PERFORMED:	04/08/2026	**DATE SUBMITTED:**	05/08/2026
NAME:	Bungag, Marc Jhowell M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA 4.1 Stacks

1. PROCEDURE OUTPUT

What procedures did you perform?

SCREENSHOT	OUTPUT

HOA 4.1 > ILOA > BUNGAG_SW4_1_ILOA.cpp > main()

```cpp
#include <iostream>
#include <stack>

int main (){
    std::stack<int> stack1;
    std::stack<int> stack2;
    std::cout<<"Testing the stack STL"<<std::endl;

    //isEmpty
    std::cout<<"Is the Stack Empty? "<<stack1.empty()<<std::endl;

    //push
    stack1.push(10);
    std::cout<<"The top of the stack is "<<stack1.top()<<std::endl;
    stack1.push(9);
    std::cout<<"The top of the stack is "<<stack1.top()<<std::endl;
    stack1.push(8);
    std::cout<<"The top of the stack is "<<stack1.top()<<std::endl;
    stack1.push(7);
    std::cout<<"The top of the stack is "<<stack1.top()<<std::endl;
    stack1.pop();
    std::cout<<"The top of the stack will be popped out"<<std::endl;
    std::cout<<"The top of the stack is "<<stack1.top()<<std::endl;
    std::cout<<"The Size of the Stack is "<<stack1.size()<<std::endl;
    stack1.emplace(6);
    std::cout<<"The top will be replaced by "<<stack1.top()<<std::endl;

    stack2.push(10);
    std::cout<<"The top of the stack is "<<stack2.top()<<std::endl;
    stack2.push(9);
    std::cout<<"The top of the stack is "<<stack2.top()<<std::endl;
    stack2.push(8);
    std::cout<<"The top of the stack is "<<stack2.top()<<std::endl;
    stack2.push(7);
    std::cout<<"The top of the stack is "<<stack2.top()<<std::endl;

    stack1.swap(stack2);
    std::cout<<"Stack 1: "<<stack1.top()<<std::endl;
    std::cout<<"Stack 2: "<<stack2.top()<<std::endl;

    return 0;
}
```

```
Testing the stack STL
Is the Stack Empty? 1
The top of the stack is 10
The top of the stack is 9
The top of the stack is 8
The top of the stack is 7
The top of the stack will be popped out
The top of the stack is 8
The Size of the Stack is 3
The top will be replaced by 6
The top of the stack is 10
The top of the stack is 9
The top of the stack is 8
The top of the stack is 7
Stack 1: 7
Stack 2: 6
```

ILO A

OBSERVATION:

Initially, the stack is empty, as indicated by the output 1 from the empty() function. After pushing the values 10, 9, 8, and 7, the last value inserted (7) becomes the top of the stack, confirming the Last In, First Out (LIFO) principle. When pop() is called, the top element (7) is removed, making 8 the new top. The size() function correctly reports that three elements remain in the stack. Finally, a second stack is created, and the swap() function exchanges the

contents of the two stacks, as shown by the final output where Stack 1 has 7 and Stack 2 has 6 on top. Overall, the results verify that the STL stack correctly performs push, pop, top, size, empty, emplace, and swap operations according to the LIFO behavior.

SCREENSHOT	OUTPUT

```
HOA 4.1 > ILOB1 > BUNGAG_SW4_1_ILOB1.cpp > ...
#include <iostream>
//global declaration
#define maxCAP 10

int stackArr[maxCAP];
int topArr = -1, newData;
//prototype functions
void push();
void pop();
void top();
bool isEmpty();
bool isFull();
void DisplayStack();

int main (){
    bool looping = true;
    int choice;
    while (looping)
    {
        std::cout<<"==================""Stack Operations""==================="<<std::endl;
        std::cout<<"Enter Your Choice"<<std::endl;
        std::cout<<"1. Push"<<std::endl;
        std::cout<<"2. Pop"<<std::endl;
        std::cout<<"3. Top"<<std::endl;
        std::cout<<"4. isEmpty"<<std::endl;
        std::cout<<"5. isFull"<<std::endl;
        std::cout<<"6. Display Stack"<<std::endl;
        std::cout<<"7. Exit"<<std::endl;
        std::cout<<"============================================================"<<std::endl;
        std::cin>>choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: top();
            break;
            case 4: std::cout<<"Is Stack Empty? "<<isEmpty()<<std::endl;
            break;
            case 5: std::cout<<"Is Stack Full? "<<isFull()<<std::endl;
            break;
            case 6: std::cout<<"Stack Displaying...";
            DisplayStack();
            std::cout<<std::endl;
            case 7: looping = false;
            break;
            default: std::cout<<"Invalid Choice"<<std::endl;
            break;
        }
```

```
==================Stack Operations==================
Enter Your Choice
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display Stack
7. Exit
============================================================
1
Enter a New Value:
12
```

```
==================Stack Operations==================
Enter Your Choice
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display Stack
7. Exit
============================================================
2
Enter the Value being Popped: 15
```

```
==================Stack Operations==================
Enter Your Choice
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display Stack
7. Exit
============================================================
3
Top Element: 14
```

```
==================Stack Operations==================
Enter Your Choice
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display Stack
7. Exit
============================================================
4
Is Stack Empty? 0
```

```
==================Stack Operations==================
Enter Your Choice
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display Stack
7. Exit
============================================================
5
Is Stack Full? 0
```

```
    }
    return 0;
}

bool isEmpty(){
    if(topArr == -1) return true;
    return false;
}
bool isFull(){
    if(topArr == maxCAP -1)return true;
    return false;
}
void push(){
    if(isFull()){
        std::cout<<"Stack Overflow"<<std::endl;
        return;
    }
    std::cout<<"Enter a New Value: "<<std::endl;
    std::cin>>newData;
    stackArr[++topArr] = newData;

}
void pop(){
    if(isEmpty()){
        std::cout<<"Stack Underflow"<<std::endl;
    }
    std::cout<<"Enter the Value being Popped: "<<stackArr[topArr]<<std::endl;
    topArr--;
}
void top(){
    if(isEmpty()){
        std::cout<<"The Stack is Empty"<<std::endl;
        return;
    }
    std::cout<<"Top Element: "<<stackArr[topArr]<<std::endl;
}
void DisplayStack(){
    for(int i = topArr; i >= 0; i--){
        std::cout<<stackArr[i]<<" ";
    }
}
```

```
==================Stack Operations==================
Enter Your Choice
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. Display Stack
7. Exit
====================================================
6
Stack Displaying...14 13 12
```

ILO B.1

OBSERVATIONS:

In this code, it demonstrates the basic operations of a stack using an array. It allows the user to interact with the stack through a menu where values can be pushed, popped, viewed, and displayed. The program also checks whether the stack is empty or full before performing operations, preventing stack overflow and underflow in most cases. As values are added and removed, the output clearly shows that the stack follows the Last In, First Out (LIFO) principle, where the most recently added element is the first one to be removed. Overall, the program provides a simple and effective way to understand how stack operations work using an array implementation.

COMPUTER ENGINEERING

QUEZON CITY

SCREENSHOT	OUTPUT
.H FILE	

HOA 4.1 > ILOB2 > C StackList.h > ...

```h
#ifndef STACKLIST_H
#define STACKLIST_H

#include<iostream>

template <typename T>
class Node{
    public:
    T data;
    Node* next;
};

template <typename T>
Node<T>* head = nullptr;

template <typename T>
void push(T newData){
    Node<T>* newNode = new Node<T>();
    newNode->data = newData;

    newNode->next = head<T>;
    head<T> = newNode;
}

template <typename T>
T pop() {
    if(head<T> == nullptr){
        std::cerr<<"Stack Underflow: Cannot Pop from an Empty Stack"<<std::endl;
    }
    Node<T>* temp = head<T>;
    T tempVal = temp->data;
    head<T> = head<T>->next;
    delete temp;
    return tempVal;
}

template<typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

template <typename T>
void Top() {
    if(head<T> == nullptr){
        std::cout<<"Stack is Empty. No Top Element"<<std::endl;
        return;
    }
    std::cout<<"Top Element: "<<head<T>->data<<std::endl;
}

template <typename T>
void DisplayAll(){
    if(head<T> == nullptr){
        std::cout<<"Stack Underflow\n"<<std::endl;
        return;
    }
    Node<T>* temp = head<T>;
    std::cout<<"Stack Elements: \n";
    while(temp != nullptr){
        std::cout<<temp->data<<" \n";
        temp = temp->next;
    }
    std::cout<<std::endl;
}
#endif
```

```
=====PUSHING 'C' 'P' 'E' ONTO THE STACK=====
Stack Elements:
E
P
C


 === POPPING ALL ELEMENTS ===
Popped: E
Popped: P
Popped: C

 === TESTING THE UNDERFLOW CONDIION ===
Stack Underflow: Cannot Pop from an Empty Stack
```

.CPP FILE

HOA 4.1 > ILOB2 > ILOB2.cpp > main()

```cpp
#include <iostream>
#include "StackList.h"

int main(){
    std::cout<<"=====PUSHING 'C' 'P' 'E' ONTO THE STACK=====\n";

    push('C');
    push('P');
    push('E');

    Top<char>;
    DisplayAll<char>();
    std::cout<<"\n === POPPING ALL ELEMENTS ===\n";

    while(!isEmpty<char>()){
        std::cout<<"Popped: "<<pop<char>()<<std::endl;
    }

    std::cout<<"\n === TESTING THE UNDERFLOW CONDIION === \n";
    pop<char>();

    return 0;
}
```

ILO B.2

OBSERVATIONS:

I observed that the program separates the implementation from the main() function, making the code more organized. The characters 'C', 'P', and 'E' are pushed onto the stack, with 'e' becoming the top element following the LIFO principle. The program then displays the stack, removes the top element using pop(), and checks if the stack is empty. The linked list implementation uses Node<T> to store data and pointers, while head<T> points to the top of the stack. The functions push(), pop(), top(), isEmpty(), and displayALL() perform the basic stack operations correctly.

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

STACKS AND ARRAYS

SCREENSHOT	OUTPUT

ILOC > ILOC_Stacks_Arrays.cpp > checkBalanced(std::string)

```cpp
#include <iostream>
#include <string>

class StackArray{
private:
    static const int MAX = 10;
    char Stack[MAX];
    int Top;

public:
    StackArray(){
        Top = -1;
    }

    bool isEmpty(){
        return Top == -1;
    }

    bool isFull(){
        return Top == MAX - 1;
    }

    void Push(char Element){
        if (!isFull()){
            Stack[++Top] = Element;
        }
    }

    char Pop(){
        if (!isEmpty()){
            return Stack[Top--];
        }
        return '\0';
    }
};

bool isMatching(char Open, char Close){
    return(Open=='('&& Close ==')')||(Open=='{'&& Close=='}')||(Open=='['&&Close==']');
}

bool checkBalanced(std::string Punctuation){
    StackArray stack;

    for(char ch:Punctuation) {
        if(ch=='('||ch =='{'||ch=='['){
            stack.Push(ch);
        }
        else if(ch==')'||ch=='}'||ch==']'){
            if (stack.isEmpty()) {
                return false;
            }
            char Open = stack.Pop();
            if(!isMatching(Open, ch)) {
                return false;
            }
        }
    }
    return stack.isEmpty();
}

int main(){
    std::string Expression;

    std::cout<<"Enter an Expression: ";
    std::getline(std::cin, Expression);

    if(checkBalanced(Expression))
        std::cout<<"The Symbols are Balanced"<<std::endl;
    else
        std::cout<<"The Symbols are NOT Balanced"<<std::endl;
    return 0;
}
```

```
Enter an Expression: (A+B)+(C-D)
The Symbols are Balanced
```

```
Enter an Expression: ((A+B)+(C-D)
The Symbols are NOT Balanced
```

```
Enter an Expression: ((A+B)+[C-D])
The Symbols are Balanced
```

```
Enter an Expression: ((A+B]+[C-D]}
The Symbols are NOT Balanced
```

STACKS AND LINKED LIST

SCREENSHOT	OUTPUT

.H FILE
ILOC > C SuppStackLinkedList.h > ...
1 #ifndef STACKLINKEDLIST_H
2 #define STACKLINKEDLIST_H
3
4 class Node {
5 public:
6 char data;
7 Node* next;
8
9 Node(char value);
10 };
11
12 class StackLinkedList {
13 private:
14 Node* top;
15
16 public:
17 StackLinkedList();
18 ~StackLinkedList();
19
20 bool isEmpty();
21 void Push(char value);
22 char Pop();
23 };
24
25 #endif
.CPP FILE
	Expression: (A+B)+(C-D)
	The Symbols are Balanced
	
	Expression: ((A+B)+(C-D)
	The Symbols are NOT Balanced
	
	Expression: ((A+B)+[C-D])
	The Symbols are Balanced
	
	Expression: ((A+B]+[C-D]}
	The Symbols are NOT Balanced

ILOC > ILOC_Stacks_LinkedList.cpp > ...

```cpp
#include <iostream>
#include "SuppStackLinkedList.h"

Node::Node(char value) {
    data = value;
    next = nullptr;
}

StackLinkedList::StackLinkedList(){
    top = nullptr;
}

StackLinkedList::~StackLinkedList(){
    while (!isEmpty()){
        Pop();
    }
}

bool StackLinkedList::isEmpty(){
    return top == nullptr;
}

void StackLinkedList::Push(char value){
    Node* newNode = new Node(value);
    newNode->next = top;
    top = newNode;
}

char StackLinkedList::Pop(){
    if (isEmpty())
        return '\0';

    Node* temp = top;
    char value = temp->data;

    top = top->next;
    delete temp;

    return value;
}

bool isMatching(char Open, char Close){
    return (Open=='('&&Close==')')||(Open=='{'&&Close=='}')||(Open=='['&&Close==']');
}

bool checkBalanced(const char Expression[]){
    StackLinkedList stack;

    for (int i = 0;Expression[i]!='\0';i++){
        char ch=Expression[i];
```

```
        if(ch=='('||ch=='{'||ch=='['){
            stack.Push(ch);
        }
        else if(ch==')'||ch=='}'||ch==']'){
            if(stack.isEmpty())
                return false;
            char Open = stack.Pop();
            if(!isMatching(Open, ch))
                return false;
        }
    }

    return stack.isEmpty();
}

int main() {

    const char* Expressions[]={
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B]+[C-D])"
    };

    int size=sizeof(Expressions)/sizeof(Expressions[0]);

    for (int i = 0; i < size; i++) {

        std::cout<<"Expression: "<<Expressions[i]<<std::endl;

        if(checkBalanced(Expressions[i]))
            std::cout<<"The Symbols are Balanced"<<std::endl;
        else
            std::cout<<"The Symbols are NOT Balanced"<<std::endl;
        std::cout<<std::endl;
    }

    return 0;
}
```

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	Expression: (A+B)+(C-D) The Symbols are Balanced	The expression is balanced because the first in the stack is an open parenthesis ”(“ and the last is a close “)”. There were no extra symbols inside the expression.
((A+B)+(C-D)	N	Expression: ((A+B)+(C-D) The Symbols are NOT Balanced	The expression is not balanced because there is one extra opening parenthesis “(“ that does not have a corresponding

			closing parenthesis ")". The stack is not empty at the end of the expression, indicating an unmatched symbol.
((A+B)+[C-D])	**Y**	`Expression: ((A+B)+[C-D])` `The Symbols are Balanced`	The expression is balanced because every opening parenthesis "(" and bracket "[" has the correct matching closing symbol ")" and "]". The symbols are also properly nested, leaving the stack empty after processing.
((A+B]+[C-D]}	**N**	`Expression: ((A+B]+[C-D]}` `The Symbols are NOT Balanced`	The expression is not balanced because the closing symbols do not match their corresponding opening symbols. A "]" attempts to close a "(", and a "}" attempts to close another "(", resulting in mismatched delimiters and an invalid expression.

CONCLUSION:

Overall, I learned how stacks work using both arrays and linked lists and how they follow the Last In, First Out (LIFO) principle. I also learned how stack operations such as push(), pop(), top(), isEmpty(), and isFull() are implemented and applied in checking balanced symbols in expressions. This activity helped me understand the importance of stacks in organizing data and solving real-world programming problems, such as expression validation.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.